

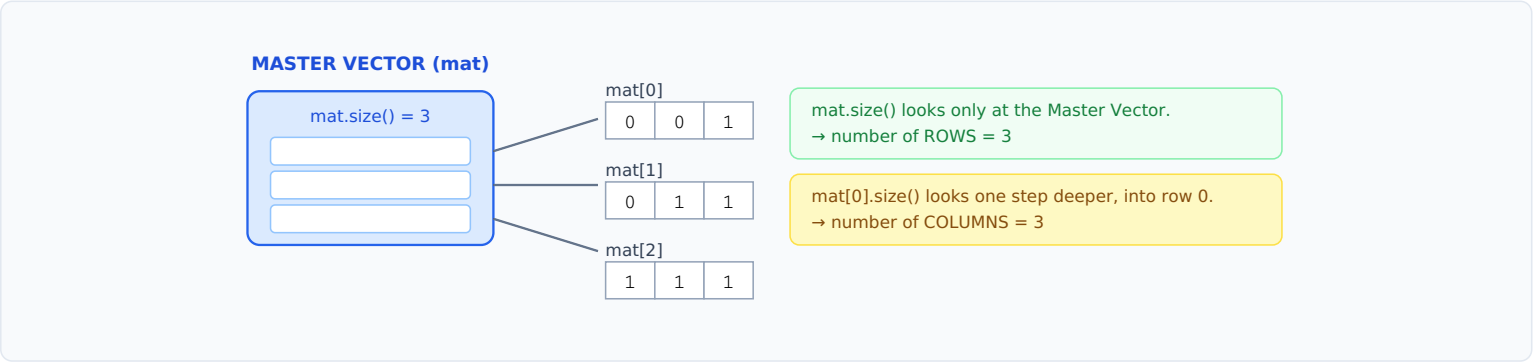
Why does mat.size()-1 work but mat[0].size()-1 crash?

Simple notes on the C++ "unsigned number" trap

In one line: `.size()` can never return a negative number. If you do math that goes below zero while still "inside" that special type, it doesn't become -1 — it jumps to a huge number instead. That huge number is then used as an array index, and the program crashes.

1 How a 2D Grid Lives in Memory

A `vector<vector<int>>` is not one solid block. It's a **Master Vector** that points to separate **Row Vectors**. For the matrix below:



Since this is a square matrix, both happen to equal 3 — but **how** C++ gets that number is very different, and that difference is the whole bug.

2 Two Kinds of Number "Wallets"

Think of a variable as a wallet that holds a number. C++ has two kinds:

The Normal Wallet — `int`

- Can hold positive numbers, zero, **and negative numbers** (like debt)
- 0 - 1 simply becomes **-1**

The Magic Wallet — `size_t`

- What `.size()` always returns. **Can never go negative**
- If math pushes it below zero, it instantly snaps to a huge number instead (about 4.29 billion)

3 What Happens When the Code Runs `col--`

Imagine the loop has already worked through the matrix and now needs to step `col` one below zero:

✗ Case A — `int col = mat[0].size() - 1;`

You reached "inside" a row, so C++ keeps treating `col` as a Magic Wallet.

`col--` → 0 - 1

Result: 4,294,967,295 (wraps around!)

The loop check `col >= 0` is still true (a huge number is ≥ 0), so it keeps going and tries to read `mat[2][4294967295]` — memory that doesn't exist.

→ **Segmentation Fault**

✓ Case B — `int col = mat.size() - 1;`

This is a simple, top-level question, so C++ safely puts the answer into your Normal Wallet (`int`).

`col--` → 0 - 1

Result: -1 (exactly as expected)

The loop check `col >= 0` is now false, so the loop stops right away, exactly when it should.

→ **Program runs correctly**

4 The Permanent Fix

Don't rely on C++ to guess the right wallet — tell it yourself by adding `(int)` in front. This makes **both** versions 100% safe:

```
int col = (int)mat[0].size() - 1; // forced to a normal int — safe
int col = (int)mat.size() - 1; // forced to a normal int — safe
```

Takeaway: Whenever a calculation using `.size()` might go below zero, cast it to `int` first. That one habit removes this entire class of crash.